Uniwersytet Łódzki
Wydział Matematyki i Informatyki

Temat ćwiczenia:

Rekurencyjna sieć neuronowa

Sprawozdanie z przedmiotu: ***Zaawansowane Metody Eksploracji***
Studia stacjonarne 2-go stopnia
Kierunek: *Analiza Danych, sem. II*
Prowadzący: *prof. dr hab. inż. Grzegorz Dudek*

Data wykonania ćwiczenia:	04.05.2025
Data złożenia sprawozdania:	11.05.2025
Data zaliczenia:	

Wykonali:	Ocena:
Dawid Sporysiak	

Nr sekcji: **3.19**

A. Cel ćwiczenia

Celem ćwiczenia jest zastosowanie sieci RNN do prognozy temperatury na podstawie posiadanych już wyników.

Opis używanej sieci LSTM (Long Short-Term Memory):

Jest to rodzaj rekurencyjnej sieci neuronowej, do modelowania zależności czasowych w danych, szczególnie przydatny do prognozowania między innymi jak w poniższym sprawozdaniu temperatury w czasie.
Sieć nazywana jest rekurencyjną, ponieważ wykorzystuje pętlę do przetwarzania danych. Każdy krok wykorzystuje informacje poprzednika, co pozwala na uszeregowanie danych czasowo. Celem tej sieci jest radzenie sobie z problemem zanikającego/zbyt szybko rosnącego gradientu, które utrudniają zwykłym sieciom naukę długoterminową. Long odnosi się do możliwości przechowywania informacji przez długi czas, razem odrzucając niepotrzebne rzeczy i aktualizując je (Short).

```
model_ext = tf.keras.Sequential([
    tf.keras.layers.LSTM(32, return_sequences=True,
input_shape=(look_back, 1)),
    tf.keras.layers.LSTM(16),
    tf.keras.layers.Dense(1)
])
model_ext.compile(optimizer='adam',loss='mse',metrics=['mae'])
history_ext = model_ext.fit(trainX_ext,
trainY,epochs=10,batch_size=200, validation_data=(testX_ext,
testY),verbose=1)
```

Zastosowane hiper parametry:
- 3 warstwy:
1) składa się z 32 neuronów, przekazuje pełną sekwencję do następnej warstwy
2) 16 neuronów
3) głęboka sieć z neuronem wyjściowym, będącym wynikiem (temperaturą)
- Optymalizator Adam
- Funkcja straty
- Metryka ‘mae’ średni błąd bezwzględny
- Liczba epok (10)
- Batch_size, czyli ilość próbek do aktualizacji wag

Jednym z elementów różniących ten model od np. CNN jest look_back określający na jak wiele informacji wstecz ma on patrzeć, look_back(3) oznacza predykcję przy pomocy trzech ostatnich danych. Mae jest lepszą metryką, by nie otrzymywać wartości kwadratowych, np. temperatura w stopniach do kwadratu.
LSTM zawiera 3 bramki:
- wejściowa, która zapisuje nowe informacje, np. ostatnie 24 pomiary temperatury, w zadaniu 7 były to 24 pomiary trzech zmiennych
- wyjściowa, która określała co będzie na wyjściu
- zapomnienia, usuwająca nieistotne dane, jak np. starsze pomiary, bądź nieistotne zmienne

B. Treść zadania

Zadanie 1

Treść:

1. Zaimportuj niezbędne moduły.

Rozwiązanie zadania:

```
#zadanie 1
import os
import pandas as pd
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error
```

Zadanie 2

Treść:

2. Wczytaj dane

```
zip_path = tf.keras.utils.get_file(
    origin='https://storage.googleapis.com/tensorflow/tf-keras-
datasets/jena_climate_2009_2016.csv.zip',
    fname='jena_climate_2009_2016.csv.zip',
    extract=True)
csv_path, _ = os.path.splitext(zip_path)
df = pd.read_csv(csv_path)

# Slice [start:stop:step], starting from index 5 take every 6th record.
df = df[5::6]

date_time = pd.to_datetime(df.pop('Date Time'), format='%d.%m.%Y %H:%M:%S')

df.head()
```

Rozwiązanie zadania:

```
#zadanie 2
csv_path = 'jena_climate_2009_2016.csv.zip'
df = pd.read_csv(csv_path)

df = df[5::6]
date_time = pd.to_datetime(df.pop('Date Time'), format='%d.%m.%Y
%H:%M:%S')

print(df.head())
```

Wynik:

```
     p (mbar)  T (degC)  Tpot (K)  Tdew (degC)  rh (%)  VPmax (mbar)  \
5      996.50     -8.05    265.38        -8.78    94.4          3.33
11     996.62     -8.88    264.54        -9.77    93.2          3.12
17     996.84     -8.81    264.59        -9.66    93.5          3.13
23     996.99     -9.05    264.34       -10.02    92.6          3.07
29     997.46     -9.63    263.72       -10.65    92.2          2.94

     VPact (mbar)  VPdef (mbar)  sh (g/kg)  H2OC (mmol/mol)  rho (g/m**3)  \
5            3.14          0.19       1.96             3.15       1307.86
11           2.90          0.21       1.81             2.91       1312.25
17           2.93          0.20       1.83             2.94       1312.18
23           2.85          0.23       1.78             2.85       1313.61
29           2.71          0.23       1.69             2.71       1317.19

     wv (m/s)  max. wv (m/s)  wd (deg)
5        0.21           0.63     192.7
11       0.25           0.63     190.3
17       0.18           0.63     167.2
23       0.10           0.38     240.0
29       0.40           0.88     157.0
```

Wniosek:

W tym zadaniu plik zip nie został pobrany bezpośrednio z linku, były przy tym nieznane problemy, więc pobrałem bezpośrednio plik i go wrzuciłem na gogle collab, przez co kod jest nieco uproszczony. Tutaj dane są przygotowane tak, by zamiast co 10 minut były wybierane te co godzinę, oraz by teksty zostały zamienione na formaty czasowe.

Zadanie 3

Treść:

3. Podaj podstawowe statystyki danych (`df.describe().transpose()`).
   Wyznacz korelacje pomiędzy szeregami czasowymi.
   Zwizualizuj powiązania pomiędzy szeregami (`sns.pairplot`).
   Zwizualizuj wszystkie szeregi czasowe w całym zakresie czasowym (2009-2017) oraz w zakresie obejmującym jeden dowolny miesiąc (patrz [*]).

Rozwiązanie zadania:

```
#zadanie 3

print("Podstawowe statystyki:")
print(df.describe().transpose())

#macierz
plt.figure(figsize=(12,8))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Macierz korelacji')
plt.show()

plt.figure(figsize=(15, 25))


for i, col in enumerate(columns_to_plot, 1):
    plt.subplot(len(columns_to_plot), 1, i)
    plt.plot(date_time, df[col], linewidth=0.5)
    plt.title(f'{col}', pad=10, loc='left')
    plt.grid(True, linestyle=':', alpha=0.3)

plt.tight_layout()
plt.suptitle('Wszystkie szeregi czasowe (2009-2017)', y=1.02)
plt.show()

#szereg czasowy miesiąca
start_date = '2015-01-01'
end_date = '2015-01-31'
month_mask = (date_time >= start_date) & (date_time <= end_date)

plt.figure(figsize=(15, 25))
for i, col in enumerate(columns_to_plot, 1):
    plt.subplot(len(columns_to_plot), 1, i)
```

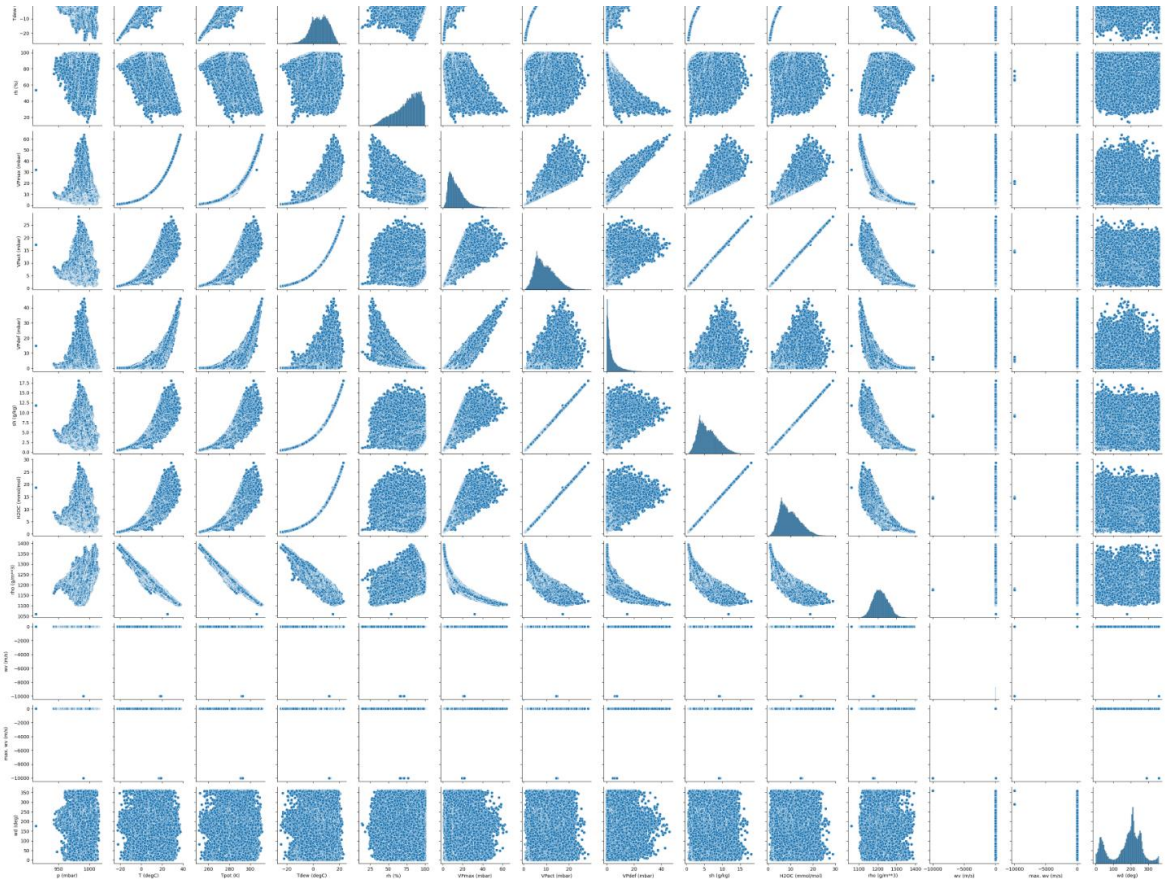
```
    plt.plot(date_time[month_mask], df.loc[month_mask, col],
linewidth=1)
    plt.title(f'{col} - styczeń 2015', pad=10, loc='left')
    plt.grid(True, linestyle=':', alpha=0.3)

plt.tight_layout()
plt.suptitle('Szeregi czasowe - styczeń 2015', y=1.02)
plt.show()

#pairplot
sns.pairplot(df)
plt.show()
```

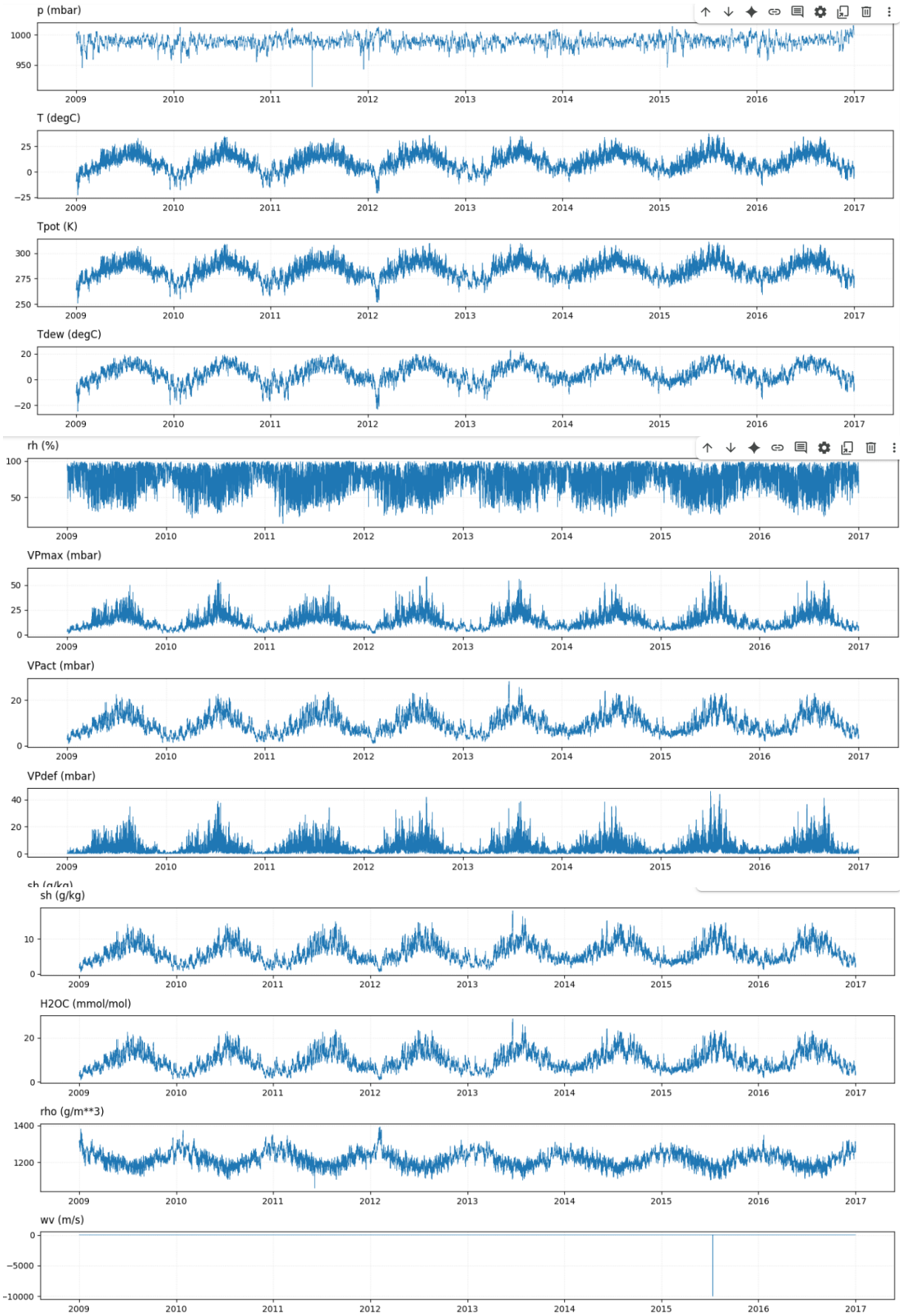
Wynik:

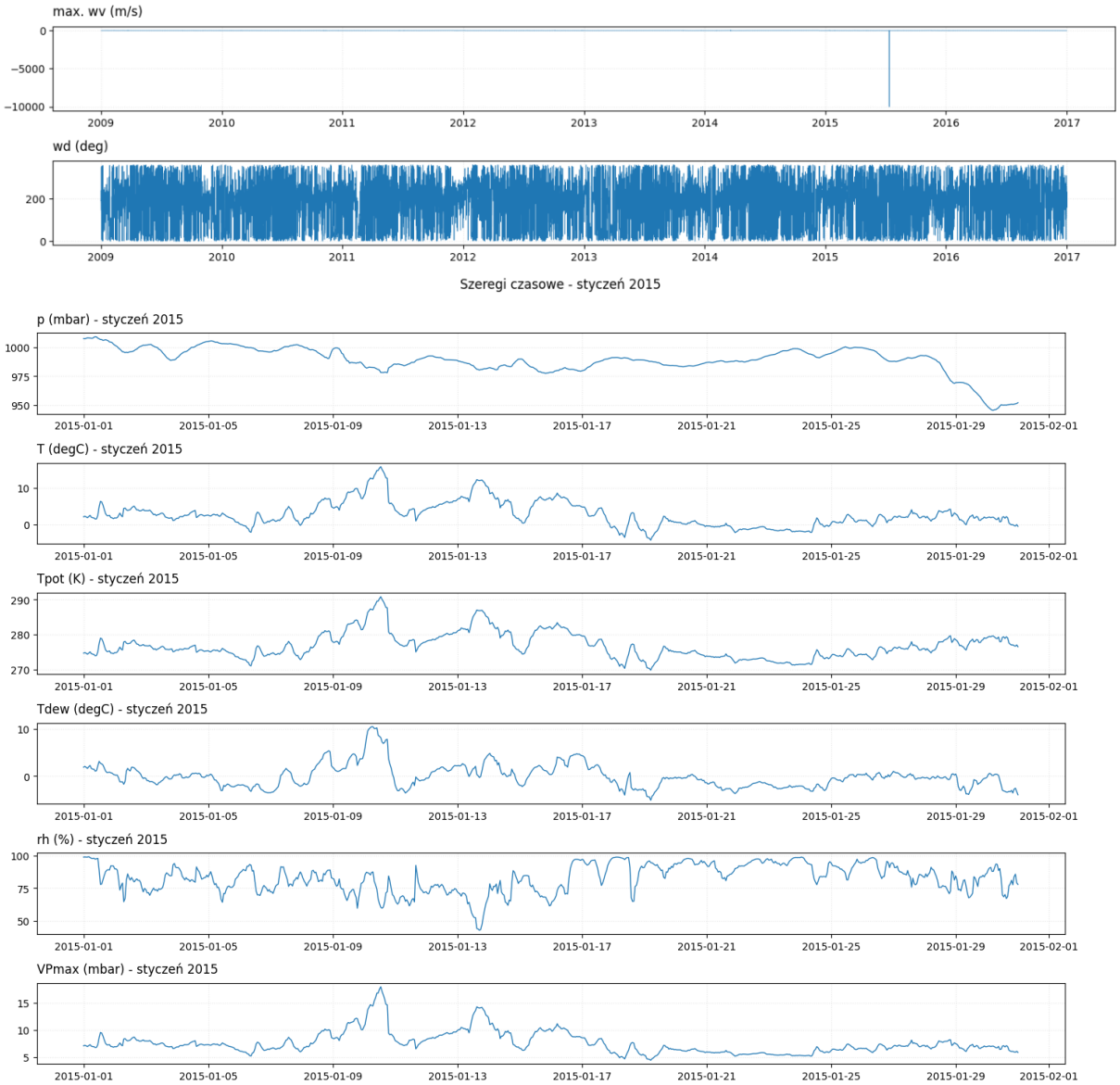
```
Podstawowe statystyki:
                    count         mean        std      min      25%      50%  \
p (mbar)          70091.0   989.212842   8.358886   913.60   984.20   989.57
T (degC)          70091.0     9.450482   8.423384   -22.76     3.35     9.41
Tpot (K)          70091.0   283.493086   8.504424   250.85   277.44   283.46
Tdew (degC)       70091.0     4.956471   6.730081   -24.80     0.24     5.21
rh (%)            70091.0    76.009788  16.474920    13.88    65.21    79.30
VPmax (mbar)      70091.0    13.576576   7.739883     0.97     7.77    11.82
VPact (mbar)      70091.0     9.533968   4.183658     0.81     6.22     8.86
VPdef (mbar)      70091.0     4.042536   4.898549     0.00     0.87     2.19
sh (g/kg)         70091.0     6.022560   2.655812     0.51     3.92     5.59
H2OC (mmol/mol)   70091.0     9.640437   4.234862     0.81     6.29     8.96
rho (g/m**3)      70091.0  1216.061232  39.974263  1059.45  1187.47  1213.80
wv (m/s)          70091.0     1.702567  65.447512 -9999.00     0.99     1.76
max. wv (m/s)     70091.0     2.963041  75.597657 -9999.00     1.76     2.98
wd (deg)          70091.0   174.789095  86.619431     0.00   125.30   198.10

                      75%      max
p (mbar)          994.720  1015.29
T (degC)           15.480    37.28
Tpot (K)          289.530   311.21
Tdew (degC)        10.080    23.06
rh (%)             89.400   100.00
VPmax (mbar)       17.610    63.77
VPact (mbar)       12.360    28.25
VPdef (mbar)        5.300    46.01
sh (g/kg)           7.800    18.07
H2OC (mmol/mol)    12.490    28.74
rho (g/m**3)     1242.765  1393.54
wv (m/s)            2.860    14.01
max. wv (m/s)       4.740    23.50
wd (deg)          234.000   360.00
```

Macierz korelacji

	p (mbar)	T (degC)	Tpot (K)	Tdew (degC)	rh (%)	VPmax (mbar)	VPact (mbar)	VPdef (mbar)	sh (g/kg)	H2OC (mmol/mol)	rho (g/m**3)	wv (m/s)	max. wv (m/s)	wd (deg)
p (mbar)	1.00	-0.05	-0.12	-0.07	-0.02	-0.03	-0.05	-0.00	-0.07	-0.07	0.31	-0.01	-0.01	-0.06
T (degC)	-0.05	1.00	1.00	0.90	-0.57	0.95	0.87	0.76	0.87	0.87	-0.96	-0.00	-0.00	0.04
Tpot (K)	-0.12	1.00	1.00	0.89	-0.57	0.95	0.87	0.76	0.87	0.87	-0.98	-0.00	-0.00	0.05
Tdew (degC)	-0.07	0.90	0.89	1.00	-0.16	0.80	0.97	0.44	0.97	0.97	-0.89	-0.01	-0.01	0.05
rh (%)	-0.02	-0.57	-0.57	-0.16	1.00	-0.62	-0.15	-0.84	-0.15	-0.15	0.51	-0.00	-0.01	-0.02
VPmax (mbar)	-0.03	0.95	0.95	0.80	-0.62	1.00	0.82	0.88	0.82	0.82	-0.90	-0.00	-0.00	-0.01
VPact (mbar)	-0.05	0.87	0.87	0.97	-0.15	0.82	1.00	0.45	1.00	1.00	-0.85	-0.01	-0.01	0.02
VPdef (mbar)	-0.00	0.76	0.76	0.44	-0.84	0.88	0.45	1.00	0.45	0.45	-0.70	0.00	0.00	-0.03
sh (g/kg)	-0.07	0.87	0.87	0.97	-0.15	0.82	1.00	0.45	1.00	1.00	-0.85	-0.01	-0.01	0.02
H2OC (mmol/mol)	-0.07	0.87	0.87	0.97	-0.15	0.82	1.00	0.45	1.00	1.00	-0.85	-0.01	-0.01	0.02
rho (g/m**3)	0.31	-0.96	-0.98	-0.89	0.51	-0.90	-0.85	-0.70	-0.85	-0.85	1.00	0.00	0.00	-0.06
wv (m/s)	-0.01	-0.00	-0.00	-0.01	-0.00	-0.00	-0.01	0.00	-0.01	-0.01	0.00	1.00	0.87	-0.02
max. wv (m/s)	-0.01	-0.00	-0.00	-0.01	-0.01	-0.00	-0.01	0.00	-0.01	-0.01	0.00	0.87	1.00	-0.01
wd (deg)	-0.06	0.04	0.05	0.05	-0.02	-0.01	0.02	-0.03	0.02	0.02	-0.06	-0.02	-0.01	1.00




p (mbar)
T (degC)
Tpot (K)
Tdew (degC)
rh (%)
VPmax (mbar)
VPact (mbar)
VPdef (mbar)
sh (g/kg)
H2OC (mmol/mol)
rho (g/m**3)
wv (m/s)
2009
2010
2011
2012
2013
2014
2015
2016
2017


max. wv (m/s)
wd (deg)
Szeregi czasowe - styczeń 2015
p (mbar) - styczeń 2015
T (degC) - styczeń 2015
Tpot (K) - styczeń 2015
Tdew (degC) - styczeń 2015
rh (%) - styczeń 2015
VPmax (mbar) - styczeń 2015

Wniosek:

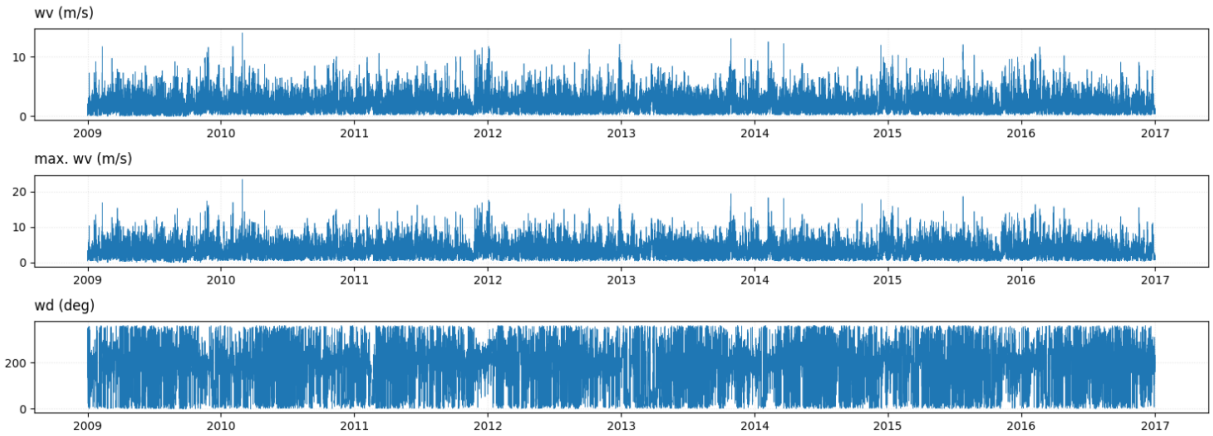
Odczytując macierz korelacji, dla tak wielu (14 wartości) można go jedynie interpretować wizualnie. Ta macierz reprezentuje korelację jednej zmiennej względem drugiej, a niemal na pewno te zmienne zależą pośrednio, i w kilka jednocześnie z różnym stopniem. Współczynniki układają się we wzór, który może sugerować, że istnieje grupowa zależność czynników.
Wykres pairplot z kolei przedstawiający 14x14=196 wykresów, już pokazuje ułożone struktury tych danych, jedne bardziej liniowe, drugie przypominające rozkład normalny, a inne układ podobny do chmury albo układ nieliniowy. Zmiennych oraz ich powiązań jest zbyt dużo, by można jednoznacznie coś wywnioskować. Dzięki df.describe widać, że zmienna wv oraz max_wv osiągają ogromne i w dodatku ujemne wartości, należało je zamienić na zera:

Poprawka do zadania:

```
for i in range(len(df)):
    if df.loc[i, 'wv (m/s)'] < 0:
        df.loc[i, 'wv (m/s)'] = 0
    if df.loc[i, 'max. wv (m/s)'] < 0:
        df.loc[i, 'max. wv (m/s)'] = 0
```

	count	mean	std	min	25%	50%
p (mbar)	70091.0	989.212842	8.358886	913.60	984.20	989.57
T (degC)	70091.0	9.450482	8.423384	-22.76	3.35	9.41
Tpot (K)	70091.0	283.493086	8.504424	250.85	277.44	283.46
Tdew (degC)	70091.0	4.956471	6.730081	-24.80	0.24	5.21
rh (%)	70091.0	76.009788	16.474920	13.88	65.21	79.30
VPmax (mbar)	70091.0	13.576576	7.739883	0.97	7.77	11.82
VPact (mbar)	70091.0	9.533968	4.183658	0.81	6.22	8.86
VPdef (mbar)	70091.0	4.042536	4.898549	0.00	0.87	2.19
sh (g/kg)	70091.0	6.022560	2.655812	0.51	3.92	5.59
H2OC (mmol/mol)	70091.0	9.640437	4.234862	0.81	6.29	8.96
rho (g/m**3)	70091.0	1216.061232	39.974263	1059.45	1187.47	1213.80
wv (m/s)	70091.0	2.130539	1.543098	0.00	0.99	1.76
max. wv (m/s)	70091.0	3.533671	2.343417	0.00	1.76	2.98
wd (deg)	70091.0	174.789095	86.619431	0.00	125.30	198.10

Wartości są już poprawne:



Zadanie 4

Treść:

4. Przygotuj dane – wytnij z oryginalnych danych 2 lata (zależnie od `nr_gr`). Przyjmij, że zbiór testowy stanowi ostatni tydzień tego okresu, a zbiór treningowy pozostałą część. Ustal rozmiar okna wejściowego (`look_back`) na 24 godz.

```
nr_gr = ?
start = int(nr_gr*300*24)
```



```
end = int(start + 730*24)
df = df[start:end]

look_back = 24
test_size = 7*24
train size = (end-start) - test size
train = df[:train_size]
test = df[train_size:]
```

Rozwiązanie zadania:

```
#zadanie 4
nr_gr = 3.19
start = int(nr_gr*300*24)
end = int(start + 730*24)
df = df[start:end]
date_time = date_time[start:end]

look_back = 24
test_size = 7*24
train_size = len(df) - test_size
train = df[:train_size]
test = df[train_size:]

print(f"Rozmiar zbioru treningowego: {len(train)}")
print(f"Rozmiar zbioru testowego: {len(test)}")
```

Wynik:

```
Rozmiar zbioru treningowego: 17352
Rozmiar zbioru testowego: 168
```

Wniosek:

Dane zostały podzielone na zbiór treningowy i testowy, zbiór treningowy to 2 lata – 7dni, które służą, jako zbiór testowy. Dane są czasowo ciągłe, muszą takie też być, by zachować sens przeprowadzania dalszych zadań. Look_back() służy tu do prognozy temperatury na następną godzinę biorą pod uwagę ostatnie 24.

Zadanie 5

Treść:

5. Dokonaj standaryzacji każdego szeregu w części treningowej i testowej (`StandardScaler()`).
   Do dalszych analiz wybierz szereg czasowy temperatury atmosferycznej (`train1 = train[:,1].reshape(-1,1), test1 = test[:,1].reshape(-1,1)`).
   Model prognostyczny będzie się uczył prognozować temperaturę na chwilę *t*+1 na podstawie temperatur w chwilach *t*-24, ..., *t* (bez wykorzystania pozostałych zmiennych).
   Zdefiniuj zbiory treningowy i testowy z wykorzystaniem funkcji `create_dataset()` (patrz wykład).

Rozwiązanie zadania:

```
#zadanie 5

#tylko temperatura
temp_scaler = StandardScaler()
train_temp_scaled = temp_scaler.fit_transform(train[['T (degC)']])
test_temp_scaled = temp_scaler.transform(test[['T (degC)']])

#treningowe
trainX = np.array([train_temp_scaled[i:i+look_back, 0] for i in
range(len(train_temp_scaled)-look_back)])
trainY = np.array([train_temp_scaled[i+look_back, 0] for i in
range(len(train_temp_scaled)-look_back)])

#testowe
testX = np.array([test_temp_scaled[i:i+look_back, 0] for i in
range(len(test_temp_scaled)-look_back)])
testY = np.array([test_temp_scaled[i+look_back, 0] for i in
range(len(test_temp_scaled)-look_back)])

trainX = trainX.reshape((trainX.shape[0], look_back, 1))
testX = testX.reshape((testX.shape[0], look_back, 1))
```

Wniosek:

Temperatura jest standaryzowana i przekształcana. X to 24 ostatnie wartości temperatury a Y to temperatura w następnej godzinie.

Zadanie 6

Treść:

6. Zbuduj model sieci RNN z jedną warstwą rekurencyjną LSTM. Skompiluj model. Przeprowadź uczenie modelu.
   Eksperymentalnie dobierz podstawowe hiperparametry: liczbę neuronów w LSTM, liczbę epok i rozmiar *batcha* (możesz przyjąć domyśle wartości: 16, 30, 500).
   Wyznacz błędy RMSE, MAE i ME (pamiętaj o destandaryzacji wyników).
   Pokaż krzywą zbieżności (`loss curve`).
   Pokaż wykresy oryginalnego szeregu i jego prognoz (część treningowa i testowa) w całym dwuletnim zakresie czasowym oraz w zakresie obejmującym ostatnie dwa tygodnie, tj. ostatni tydzień treningowy i tydzień testowy (na oddzielnym rysunku).

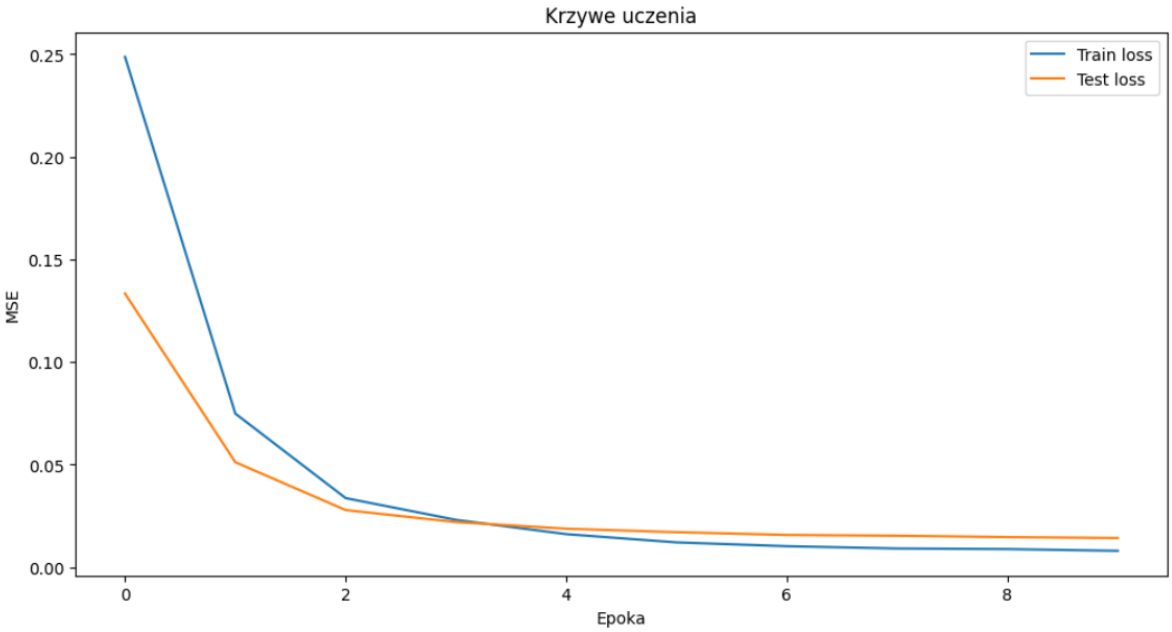
Rozwiązanie zadania:

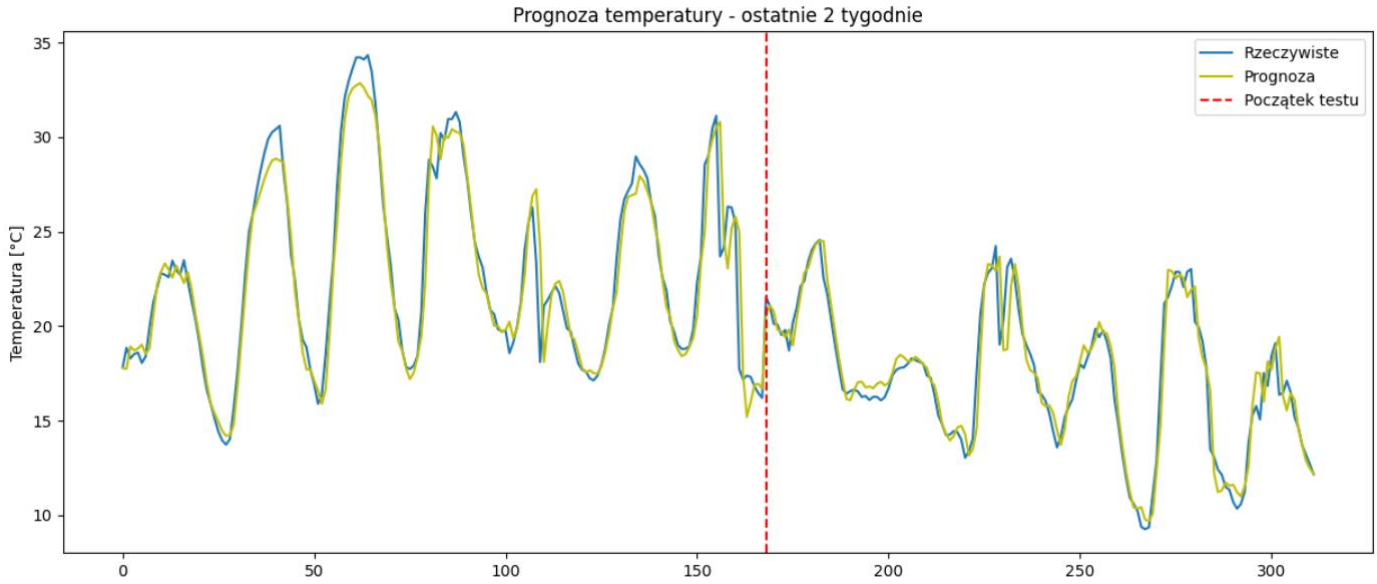
```
#zadanie 6

#model LSTM
model = tf.keras.Sequential([
    tf.keras.layers.LSTM(32, return_sequences=True,
input_shape=(look_back, 1)),
    tf.keras.layers.LSTM(16),
    tf.keras.layers.Dense(1)
])

model.compile(optimizer='adam', loss='mse',metrics=['mae'])

#trening
history = model.fit(trainX,
trainY,epochs=10,batch_size=200,validation_data=(testX,
testY),verbose=1)

trainPredict = model.predict(trainX)
testPredict = model.predict(testX)

#destandaryzacja
trainPredict = temp_scaler.inverse_transform(trainPredict)
trainY_orig = temp_scaler.inverse_transform(trainY.reshape(-1, 1))
testPredict = temp_scaler.inverse_transform(testPredict)
testY_orig = temp_scaler.inverse_transform(testY.reshape(-1, 1))
```

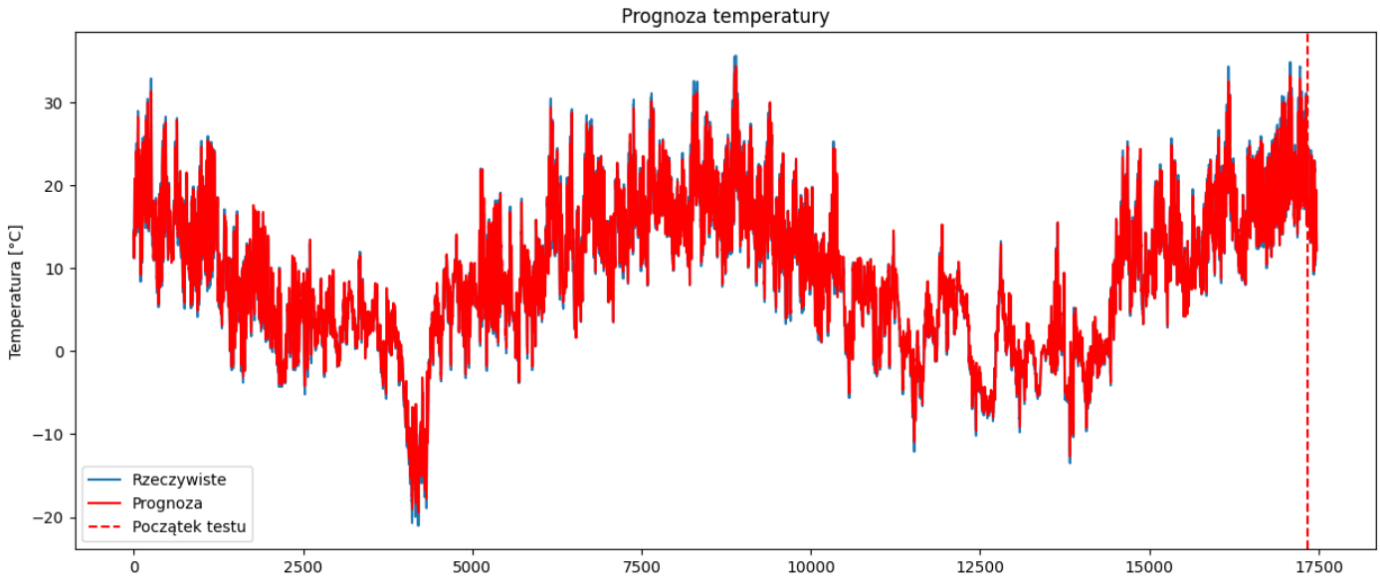
```
#RMSE, MAE, ME
train_rmse = np.sqrt(mean_squared_error(trainY_orig, trainPredict))
test_rmse = np.sqrt(mean_squared_error(testY_orig, testPredict))
train_mae = mean_absolute_error(trainY_orig, trainPredict)
test_mae = mean_absolute_error(testY_orig, testPredict)
train_me = np.mean(trainY_orig - trainPredict)
test_me = np.mean(testY_orig - testPredict)

print("\nMetryki:")
print(f"TRENINGOWY: RMSE = {train_rmse:.2f}°C, MAE = {train_mae:.2f}°C,
ME = {train_me:.2f}°C")
print(f"TESTOWY:  RMSE = {test_rmse:.2f}°C, MAE = {test_mae:.2f}°C, ME
= {test_me:.2f}°C")

#krzywa uczenia
plt.figure(figsize=(12, 6))
plt.plot(history.history['loss'], label='Train loss')
plt.plot(history.history['val_loss'], label='Test loss')
plt.title('Krzywe uczenia')
plt.ylabel('MSE')
plt.xlabel('Epoka')
plt.legend()
plt.show()

#pełna prognoza 2 lat
plt.figure(figsize=(15, 6))
full_range = np.concatenate([trainY_orig, testY_orig])
full_predict = np.concatenate([trainPredict, testPredict])
plt.plot(full_range, label='Rzeczywiste')
plt.plot(full_predict, label='Prognoza', alpha=1,color='r')
plt.axvline(x=len(trainY_orig), color='r', linestyle='--',
label='Początek testu')
plt.title('Prognoza temperatury na 2 lata')
plt.ylabel('Temperatura [°C]')
plt.legend()
plt.show()

#ostatnie 2 tygodnie
plt.figure(figsize=(15, 6))
last_two_weeks = np.concatenate([trainY_orig[-7*24:], testY_orig])
last_two_weeks_pred = np.concatenate([trainPredict[-7*24:],
testPredict])
plt.plot(last_two_weeks, label='Rzeczywiste')
plt.plot(last_two_weeks_pred, label='Prognoza', alpha=1, color = 'y')
plt.axvline(x=7*24, color='r', linestyle='--', label='Początek testu')
plt.title('Ostatnie 2 tygodnie')
plt.ylabel('Temperatura [°C]')
plt.legend()
plt.show()
```

Wynik:

```
87/87 ━━━━━━━━━━━━━━━━━━━━ 8s 49ms/step - loss: 0.4101 - mae: 0.4806 - val_loss: 0.1334 - val_mae: 0.2914
Epoch 2/10
87/87 ━━━━━━━━━━━━━━━━━━━━ 6s 59ms/step - loss: 0.0987 - mae: 0.2382 - val_loss: 0.0512 - val_mae: 0.1827
Epoch 3/10
87/87 ━━━━━━━━━━━━━━━━━━━━ 11s 67ms/step - loss: 0.0375 - mae: 0.1456 - val_loss: 0.0279 - val_mae: 0.1179
Epoch 4/10
87/87 ━━━━━━━━━━━━━━━━━━━━ 7s 34ms/step - loss: 0.0251 - mae: 0.1189 - val_loss: 0.0220 - val_mae: 0.1047
Epoch 5/10
87/87 ━━━━━━━━━━━━━━━━━━━━ 6s 42ms/step - loss: 0.0176 - mae: 0.0983 - val_loss: 0.0188 - val_mae: 0.0965
Epoch 6/10
87/87 ━━━━━━━━━━━━━━━━━━━━ 4s 40ms/step - loss: 0.0126 - mae: 0.0819 - val_loss: 0.0171 - val_mae: 0.0916
Epoch 7/10
87/87 ━━━━━━━━━━━━━━━━━━━━ 3s 32ms/step - loss: 0.0104 - mae: 0.0740 - val_loss: 0.0158 - val_mae: 0.0861
Epoch 8/10
87/87 ━━━━━━━━━━━━━━━━━━━━ 3s 37ms/step - loss: 0.0093 - mae: 0.0693 - val_loss: 0.0154 - val_mae: 0.0863
Epoch 9/10
87/87 ━━━━━━━━━━━━━━━━━━━━ 4s 40ms/step - loss: 0.0086 - mae: 0.0661 - val_loss: 0.0147 - val_mae: 0.0824
Epoch 10/10
87/87 ━━━━━━━━━━━━━━━━━━━━ 3s 32ms/step - loss: 0.0085 - mae: 0.0662 - val_loss: 0.0143 - val_mae: 0.0833
542/542 ━━━━━━━━━━━━━━━━━━━━ 4s 6ms/step
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step

Metryki:
TRENINGOWY: RMSE = 0.78°C, MAE = 0.55°C, ME = -0.08°C
TESTOWY:  RMSE = 1.06°C, MAE = 0.74°C, ME = -0.11°C
```







Wniosek:

W tym zadaniu użyto sieci LSTM opisanej na początku sprawozdania. Po destandaryzacji MAE otrzymaliśmy wynik 0,74 stopni na zbiorze testowym, czyli średnio różnica pomiędzy prognozą a wartością rzeczywistą wynosiła 0,74. Patrząc na wykres można stwierdzić, że jest to raczej akceptowalna różnica. Przy bardziej złożonej sieci wynik byłby lepszy, otrzymany tutaj takim na pewno nie jest.

Zadanie 7

Treść:

7. Czy dodanie dodatkowych zmiennych wejściowych (egzogenicznych) poprawi wyniki? Sprawdź to eksperymentalnie, powtarzając p. 6.

   Wskazówka:

   - rozszerz dane wejściowe o dodatkowe atrybuty, np. ciśnienie atmosferyczne `p (mbar)` i gęstość powietrza `rho (g/m**3)` (możesz użyć innych zmiennych). W tym celu użyj funkcji `create_dataset()` dla tych nowych zmiennych, podobnie jak wcześniej do danych temperaturowych. Otrzymane w wyniku zbiory `trainX0` i `testX0` oraz `trainX10` i `testX10` połącz ze zbiorami otrzymanymi wcześniej dla temperatury (`trainX1` i `testX1`):

```
trainXa = np.concatenate((trainX1,trainX0,trainX10),axis=1)
testXa = np.concatenate((testX1,testX0,testX10),axis=1)
```

Rozwiązanie zadania:

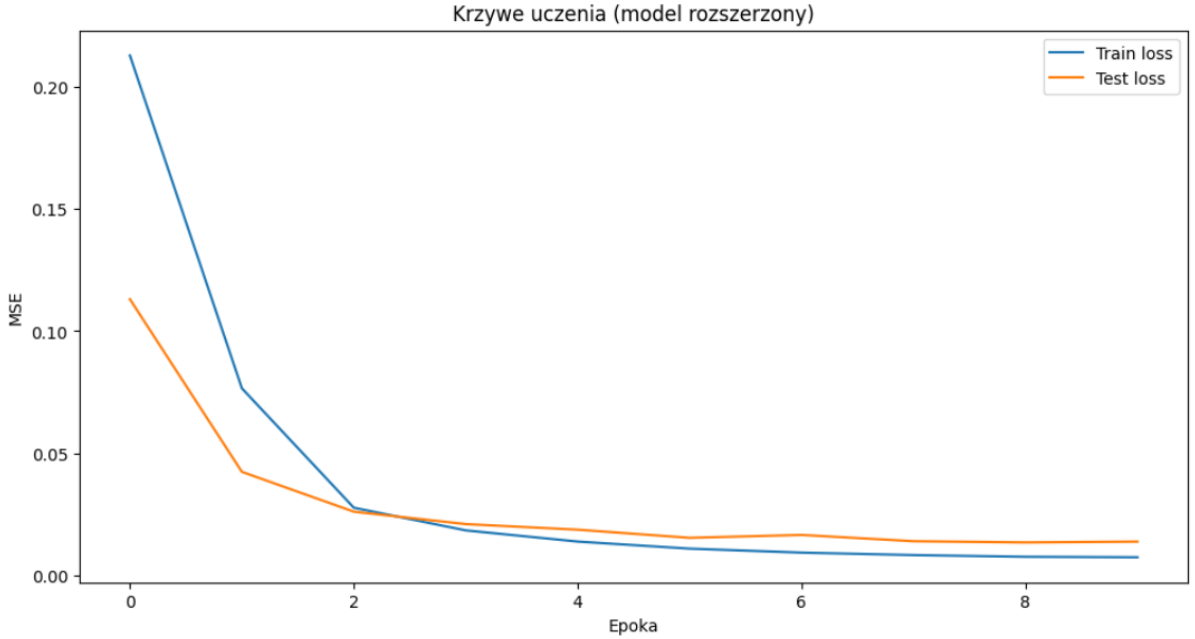
```
#zadanie 7

#standaryzacja
p_scaler = StandardScaler()
rho_scaler = StandardScaler()

train_p_scaled = p_scaler.fit_transform(train[['p (mbar)']])
train_rho_scaled = rho_scaler.fit_transform(train[['rho (g/m**3)']])
test_p_scaled = p_scaler.transform(test[['p (mbar)']])
test_rho_scaled = rho_scaler.transform(test[['rho (g/m**3)']])

def create_dataset_multi(X, look_back=1):
    Xs = []
    for i in range(len(X)-look_back):
        Xs.append(X[i:(i+look_back), 0])
    return np.expand_dims(np.array(Xs), axis=-1)

trainX_p = create_dataset_multi(train_p_scaled, look_back)
trainX_rho = create_dataset_multi(train_rho_scaled, look_back)
testX_p = create_dataset_multi(test_p_scaled, look_back)
testX_rho = create_dataset_multi(test_rho_scaled, look_back)

#łączenie danych
trainX_ext = np.concatenate([trainX, trainX_p, trainX_rho], axis=2)
testX_ext = np.concatenate([testX, testX_p, testX_rho], axis=2)
#model (z zadania 6)
```

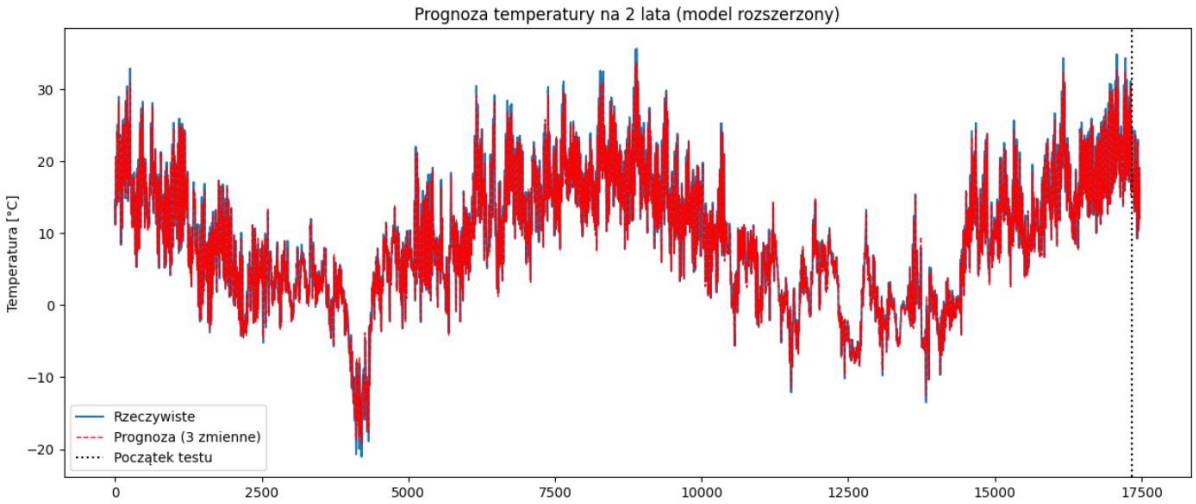
```
model_ext = tf.keras.Sequential([
    tf.keras.layers.LSTM(32, return_sequences=True,
input_shape=(look_back, 3)),
    tf.keras.layers.LSTM(16),
    tf.keras.layers.Dense(1)
])
model_ext.compile(optimizer='adam',loss='mse',metrics=['mae'])
history_ext = model_ext.fit(trainX_ext,
trainY,epochs=10,batch_size=200, validation_data=(testX_ext,
testY),verbose=1)

trainPredict_ext = model_ext.predict(trainX_ext)
testPredict_ext = model_ext.predict(testX_ext)

#destandaryzacja
trainPredict_ext = temp_scaler.inverse_transform(trainPredict_ext)
testPredict_ext = temp_scaler.inverse_transform(testPredict_ext)

#metryki
train_rmse_ext = np.sqrt(mean_squared_error(trainY_orig,
trainPredict_ext))
test_rmse_ext = np.sqrt(mean_squared_error(testY_orig,
testPredict_ext))
train_mae_ext = mean_absolute_error(trainY_orig, trainPredict_ext)
test_mae_ext = mean_absolute_error(testY_orig, testPredict_ext)
train_me_ext = np.mean(trainY_orig - trainPredict_ext)
test_me_ext = np.mean(testY_orig - testPredict_ext)

print("\nMetryki modelu rozszerzonego:")
print(f"TRENINGOWY: RMSE = {train_rmse_ext:.2f}°C, MAE =
{train_mae_ext:.2f}°C, ME = {train_me_ext:.2f}°C")
print(f"TESTOWY:    RMSE = {test_rmse_ext:.2f}°C, MAE =
{test_mae_ext:.2f}°C, ME = {test_me_ext:.2f}°C")

#krzywa uczenia
plt.figure(figsize=(12, 6))
plt.plot(history_ext.history['loss'], label='Train loss')
plt.plot(history_ext.history['val_loss'], label='Test loss')
plt.title('Krzywe uczenia (3 zmienne)')
plt.ylabel('MSE')
plt.xlabel('Epoka')
plt.legend()
plt.show()

#2 lata prognozy
plt.figure(figsize=(15, 6))
plt.plot(full_range, label='Rzeczywiste', linewidth=1.5)
plt.plot(np.concatenate([trainPredict_ext, testPredict_ext]),
label='Prognoza (3 zmienne)',linewidth=1,color='red')
```

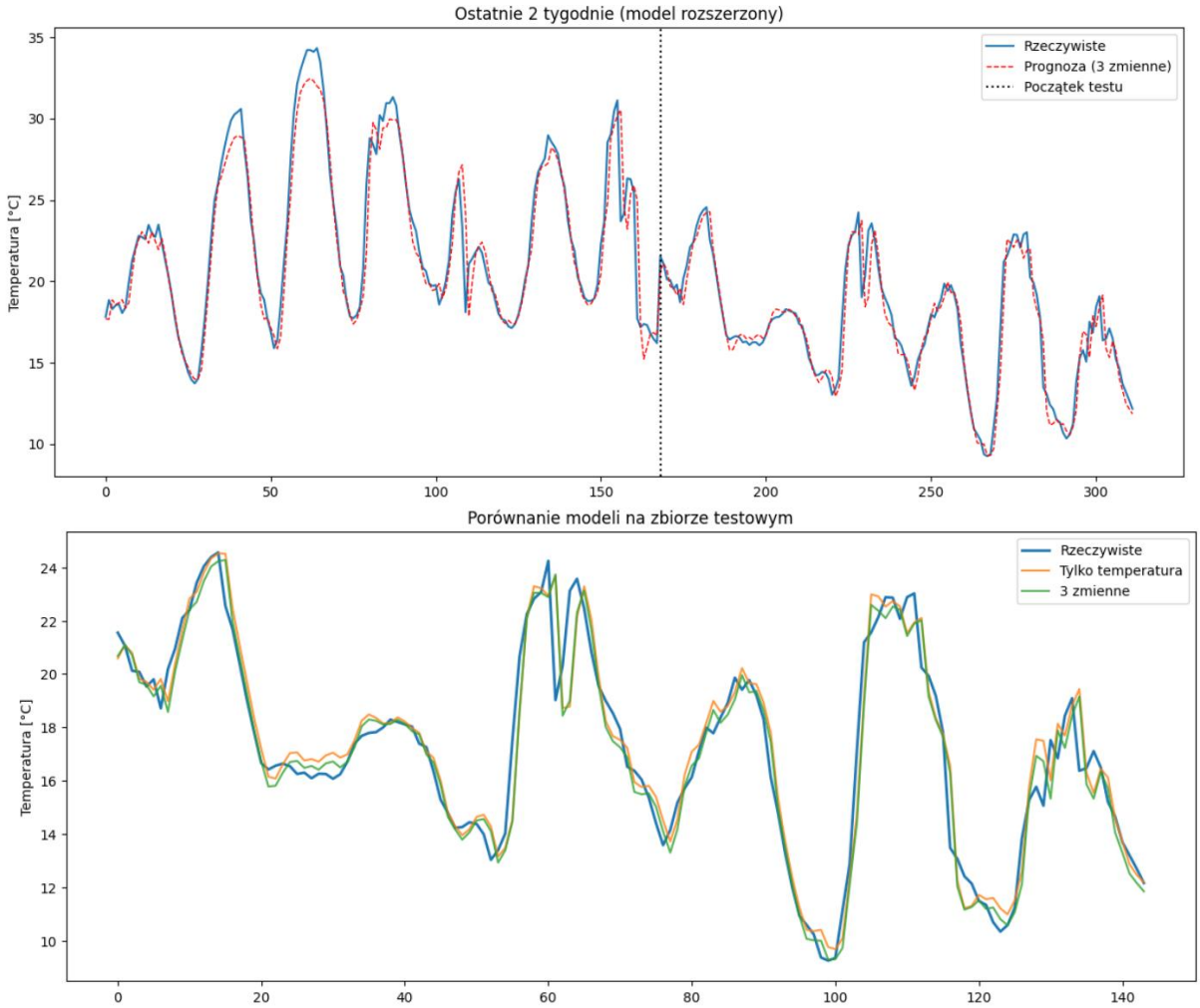
```
plt.axvline(x=len(trainY_orig), color='black', linestyle=':',
label='Początek testu')
plt.title('Prognoza temperatury na 2 lata (model rozszerzony)')
plt.ylabel('Temperatura [°C]')
plt.legend()
plt.show()

#ostatnie 2 tygodnie
plt.figure(figsize=(15, 6))
last_two_weeks_pred_ext = np.concatenate([trainPredict_ext[-7*24:],
testPredict_ext])
plt.plot(last_two_weeks, label='Rzeczywiste', linewidth=1.5)
plt.plot(last_two_weeks_pred_ext, label='Prognoza z 3
zmiennymi',linewidth=1,color='red')
plt.axvline(x=7*24, color='black', linestyle=':', label='Początek
testu')
plt.title('Ostatnie 2 tygodnie (model rozszerzony)')
plt.ylabel('Temperatura [°C]')
plt.legend()
plt.show()

#porównanie modeli dla tygodnia
plt.figure(figsize=(15, 6))
plt.plot(testY_orig, label='Rzeczywiste', linewidth=2)
plt.plot(testPredict, label='Tylko temperatura', alpha=0.8)
plt.plot(testPredict_ext, label='3 zmienne', alpha=0.8)
plt.title('Porównanie wszystkich modeli na zbiorze testowym')
plt.ylabel('Temperatura [°C]')
plt.legend()
plt.show()
```

Wynik:

```
87/87 ━━━━━━━━━━━━━━━━━━━━ 19s 36ms/step - loss: 0.3273 - mae: 0.4292 - val_loss: 0.1130 - val_mae: 0.2682
Epoch 2/10
87/87 ━━━━━━━━━━━━━━━━━━━━ 4s 41ms/step - loss: 0.0980 - mae: 0.2403 - val_loss: 0.0425 - val_mae: 0.1710
Epoch 3/10
87/87 ━━━━━━━━━━━━━━━━━━━━ 4s 31ms/step - loss: 0.0324 - mae: 0.1337 - val_loss: 0.0262 - val_mae: 0.1266
Epoch 4/10
87/87 ━━━━━━━━━━━━━━━━━━━━ 3s 30ms/step - loss: 0.0200 - mae: 0.1050 - val_loss: 0.0211 - val_mae: 0.1095
Epoch 5/10
87/87 ━━━━━━━━━━━━━━━━━━━━ 3s 30ms/step - loss: 0.0151 - mae: 0.0902 - val_loss: 0.0188 - val_mae: 0.1025
Epoch 6/10
87/87 ━━━━━━━━━━━━━━━━━━━━ 5s 30ms/step - loss: 0.0113 - mae: 0.0778 - val_loss: 0.0155 - val_mae: 0.0874
Epoch 7/10
87/87 ━━━━━━━━━━━━━━━━━━━━ 3s 30ms/step - loss: 0.0098 - mae: 0.0706 - val_loss: 0.0167 - val_mae: 0.0957
Epoch 8/10
87/87 ━━━━━━━━━━━━━━━━━━━━ 3s 30ms/step - loss: 0.0083 - mae: 0.0652 - val_loss: 0.0141 - val_mae: 0.0852
Epoch 9/10
87/87 ━━━━━━━━━━━━━━━━━━━━ 3s 34ms/step - loss: 0.0082 - mae: 0.0634 - val_loss: 0.0136 - val_mae: 0.0790
Epoch 10/10
87/87 ━━━━━━━━━━━━━━━━━━━━ 3s 39ms/step - loss: 0.0076 - mae: 0.0611 - val_loss: 0.0139 - val_mae: 0.0846
542/542 ━━━━━━━━━━━━━━━━━━━━ 4s 6ms/step
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step

Metryki modelu rozszerzonego:
TRENINGOWY: RMSE = 0.81°C, MAE = 0.58°C, ME = 0.28°C
TESTOWY:    RMSE = 1.03°C, MAE = 0.70°C, ME = 0.16°C
```


Krzywe uczenia (model rozszerzony)
Train loss
Test loss
MSE
Epoka
0.20
0.15
0.10
0.05
0.00
0
2
4
6
8


Prognoza temperatury na 2 lata (model rozszerzony)
Temperatura [°C]
Rzeczywiste
Prognoza (3 zmienne)
Początek testu
30
20
10
0
−10
−20
0
2500
5000
7500
10000
12500
15000
17500



Wniosek:

Po dodaniu trzech zmiennych wynik nie poprawił się znacząco. Oba przewidywania są całkiem zbliżone do rzeczywistych wyników zarówno na zbiorze treningowym jak i testowym. Użycie dodatkowych zmiennych wiele nie pomogło. Być może nie są one odpowiednim rozwiązaniem, i należałoby przetestować ich możliwe kombinacje w celu uzyskania lepszego wyniku.

Wnioski końcowe:

Główną część stanowiły dwa ostatnie zadania, w których została wykorzystana sieć LSTM. Do tego rodzaju testu była ona dobrym rozwiązaniem. Otrzymane wyniki zarówno na zbiorze treningowym jak i testowym w obu zadaniach nie były złe, ale mogłyby one być lepsze przy odpowiednim rozwinięciu sieci, oraz kombinacji parametrów wejściowych, jak i chociażby param_grid w celu wybrania najlepszej siatki hiperparametrów, jak np. większy batch, większa liczba epok, dodatkowe warstwy itp. Prognoza na dodatkowy tydzień okazała się całkiem trafna. Df.describe() w zadaniu 3 okazało się też przydatnym narzędziem, ponieważ wychwycony dzięki niemu został błędny pomiar powodujący duże zaburzenie wykresu dla dwóch zmiennych, dzięki czemu zostały one poprawione.